

Ripasso Completo — Esame di Riparazione Informatica

HTML · Bootstrap · JavaScript · DOM · CSV · AJAX/Fetch · Cache Remota

Indice

1. HTML
2. Bootstrap
3. JavaScript Base
4. Array e Dizionari
5. JavaScript nel Browser (DOM)
6. Evento, Azione, Reazione
7. CSV
8. AJAX e Fetch
9. Cache Remota

Sezione 1 — HTML

Struttura base di una pagina HTML

Ogni pagina HTML parte da uno scheletro fisso: il **DOCTYPE** dichiara che il documento è HTML5, il tag **<html>** racchiude tutta la pagina, **<head>** contiene informazioni non visibili (titolo, collegamenti a CSS), **<body>** contiene tutto ciò che si vede nel browser.

```
<!DOCTYPE html>

<html lang="it">

<head>

<meta charset="UTF-8">

<title>La mia pagina</title>

</head>

<body>

<h1>Ciao mondo!</h1>

</body>

</html>
```

Tag principali

Ogni tag ha uno scopo semantico preciso: dice al browser cosa significa quel contenuto.

Tag / attributo	A cosa serve
h1 - h6	Titoli, dal più importante (h1) al meno importante (h6).
p	Un paragrafo di testo.
a	Link a un'altra pagina o sezione (attributo href).
img	Immagine (attributo src per il percorso, alt per il testo alternativo).
ul / ol / li	Lista non ordinata (puntata) / ordinata (numerata) / singolo elemento.
table / thead / tbody / tr / th / td	Tabella: intestazione, corpo, riga, cella di intestazione, cella dati.
form	Contenitore che raggruppa i campi di un modulo da inviare o leggere con JS.
input	Campo di inserimento dati (il tipo cambia comportamento, vedi sotto).
select	Menu a tendina con opzioni (<option>).

Tag / attributo	A cosa serve
textarea	Area di testo multi-riga.
button	Pulsante cliccabile, spesso collegato a un evento JavaScript.
div	Contenitore generico a blocco, senza significato semantico proprio.
span	Contenitore generico in linea, senza significato semantico proprio.

I tipi di input più usati

Tag / attributo	A cosa serve
text	Testo libero su una riga (nome, cognome, ecc.).
number	Solo numeri, con frecce su/giù per incrementare.
email	Testo con validazione automatica del formato email.
password	Testo nascosto con puntini al posto dei caratteri.
date	Selettore di data con calendario integrato dal browser.
checkbox	Casella di spunta, vero/falso (attributo checked).

L'attributo id

id è un identificatore **unico** nella pagina: nessun altro elemento può avere lo stesso id. Serve principalmente come **gancio per JavaScript**: permette di recuperare esattamente quell'elemento con `document.querySelector('#id')` e leggerne o modificarne il contenuto. Senza un id (o un selettore equivalente) JavaScript non ha modo di sapere a quale elemento riferirsi.

```
<input type="text" id="nomeStudente">

<script>

const campo = document.querySelector("#nomeStudente");

console.log(campo.value);

</script>
```

div vs span

div è un elemento **a blocco**: occupa sempre tutta la larghezza disponibile e va automaticamente a capo prima e dopo. Si usa per raggruppare sezioni intere di pagina (un blocco form, una card, un contenitore).

span è un elemento **in linea**: occupa solo lo spazio del suo contenuto e resta sulla stessa riga del testo circostante. Si usa per evidenziare o stilizzare una piccola parte di testo dentro un paragrafo o un titolo.

```
<div>Questo è un blocco, va a capo prima e dopo</div>
```

```
<p>Testo con una parola <span style="color:red">rossa</span> in  
linea.</p>
```

Sezione 2 — Bootstrap

Come collegare Bootstrap dalla CDN

Bootstrap si collega senza scaricare nulla: basta un tag `<link>` nell'`<head>` per il CSS e uno `<script>` prima di `</body>` per la parte JavaScript (necessaria solo per componenti interattivi come dropdown e modali).

```
<head>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/cs
s/bootstrap.min.css"

rel="stylesheet">

</head>

<body>

...

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/j
s/bootstrap.bundle.min.js"></script>

</body>
```

Il sistema a 12 colonne

La griglia Bootstrap segue sempre la struttura **container > row > col**. Ogni riga (row) è divisa in **12 colonne**: la somma dei numeri usati nelle classi col-N di una stessa riga deve fare 12 (es. col-4 + col-8, oppure col-6 + col-6).

```
<div class="container">

<div class="row">

<div class="col-4">Colonna stretta</div>

<div class="col-8">Colonna larga</div>

</div>

</div>
```

I breakpoint responsive

Le classi col-* cambiano nome a seconda della larghezza minima dello schermo a cui si applicano: questo permette layout diversi su mobile, tablet e desktop.

Classe	Breakpoint (larghezza minima)
col	Sempre attiva, nessun breakpoint (mobile first)
col-sm-*	≥ 576px (smartphone orizzontale)

Classe	Breakpoint (larghezza minima)
col-md-*	≥ 768px (tablet)
col-lg-*	≥ 992px (desktop)

Spacing: margin e padding

Le classi di spacing seguono una logica fissa: **m** (margin, esterno) o **p** (padding, interno) + **lato** (t=top, b=bottom, s=start/sinistra, e=end/destra, x=orizzontale, y=verticale) + **numero da 0 a 5** (0 = nessuno spazio, 5 = massimo).

```
<div class="mt-3 mb-2 px-4 py-2">

<!-- margin-top livello 3, margin-bottom livello 2,
padding orizzontale livello 4, padding verticale livello 2 -->

</div>
```

Classi pulsanti

Base **btn** + colore: btn-primary, btn-success, btn-danger, btn-warning.
Versione contornata: btn-outline-* (stesso schema colori, sfondo trasparente).

```
<button class="btn btn-primary">Salva</button>

<button class="btn btn-outline-danger">Elimina</button>
```

Classi form

Tag / attributo	A cosa serve
form-label	Etichetta di un campo form.
form-control	Stile standard per input di testo, number, email, ecc.
form-select	Stile standard per un menu a tendina (select).
form-check	Contenitore per checkbox/radio con etichetta allineata.

Classi tabella

Tag / attributo	A cosa serve
table	Classe base, obbligatoria per lo stile Bootstrap sulla tabella.
table-striped	Righe con sfondo alternato per leggibilità.
table-hover	Evidenzia la riga sotto il puntatore del mouse.
table-bordered	Aggiunge bordi a tutte le celle.
table-dark	Sfondo scuro (spesso usata su thead).
badge	Etichetta colorata compatta, usata dentro le celle (es. Promosso/Bocciato).

Classi card

Tag / attributo	A cosa serve
card	Contenitore principale con bordo e ombra leggera.
card-header	Intestazione della card.
card-body	Corpo principale della card, contiene titolo e testo.
card-title	Titolo dentro il card-body.
card-text	Paragrafo di testo dentro il card-body.

Classi alert

Base **alert** + colore: alert-success (verde, ok), alert-danger (rosso, errore), alert-warning (giallo, attenzione), alert-info (azzurro, informazione).

```
<div class="alert alert-success">Operazione riuscita!</div>  
  
<div class="alert alert-danger">Errore: campo  
obbligatorio.</div>
```

Classi utility utili

Tag / attributo	A cosa serve
d-flex	Attiva il layout flessibile (flexbox) sul contenitore.
align-items-center	Allinea verticalmente al centro gli elementi flex.
justify-content-center	Allinea orizzontalmente al centro gli elementi flex.
min-vh-100	Altezza minima pari al 100% dell'altezza della finestra (viewport).
w-100	Larghezza al 100% del contenitore padre.
text-center	Centra il testo orizzontalmente.

Sezione 3 — JavaScript Base

Variabili: let e const

const si usa di default: dichiara una variabile che non verrà mai riassegnata. **let** si usa solo quando il valore deve cambiare nel tempo (es. un contatore). Usare const quando possibile rende chiaro a chi legge il codice cosa può cambiare e cosa no.

```
const nome = "Luca"; // non cambierà più  
  
let punteggio = 0; // cambierà con il gioco  
  
punteggio = punteggio + 10;
```

Tipi di dato

Tag / attributo	A cosa serve
stringa	Testo tra virgolette: "ciao" oppure 'ciao'.
numero	Interi o decimali, senza virgolette: 10, 3.14.
booleano	Solo due valori possibili: true o false.
undefined	Una variabile dichiarata ma senza valore assegnato.
null	Un valore assente impostato volontariamente dal programmatore.

Operatori

Aritmetici: + - * / % (resto della divisione). **Di confronto:** === confronta valore E tipo (da preferire sempre), == confronta solo il valore convertendo i tipi (da evitare, causa risultati sorprendenti). **Logici:** && (E, vero se entrambi veri), || (O, vero se almeno uno vero), ! (negazione).

```
"10" === 10 // false, tipo diverso (stringa vs numero)  
  
"10" == 10 // true, ma va evitato  
  
let eta = 20;  
  
if (eta >= 18 && eta < 65) {  
  console.log("Maggiorenne e non pensionato");  
}
```


if / else

```
const voto = 5;

if (voto >= 6) {
  console.log("Promosso");
} else {
  console.log("Bocciato");
}
```

Ciclo for classico e for...of

Il **for** classico si usa quando servono l'indice e il controllo preciso sull'iterazione. **for...of** è più semplice quando serve solo il valore di ogni elemento, senza bisogno dell'indice.

```
const voti = [6, 7, 5, 9];

for (let i = 0; i < voti.length; i++) {
  console.log(i, voti[i]);
}

for (const voto of voti) {
  console.log(voto);
}
```

while

Si usa quando il numero di ripetizioni dipende da una condizione, non è noto in anticipo.

```
let tentativi = 0;

while (tentativi < 3) {
  console.log("Tentativo numero", tentativi);
  tentativi++;
}
```

Funzioni: classiche e arrow

Stessa funzione scritta nei due modi equivalenti:

```
// Modo classico

function somma(a, b) {
  return a + b;
}

// Arrow function

const somma = (a, b) => {
  return a + b;
};

// Arrow function compatta (return implicito)

const somma = (a, b) => a + b;
```

Template literal e concatenazione

```
const nome = "Anna";
const voto = 8;

// Template literal (preferito, più leggibile)
console.log(`${nome} ha preso ${voto}`);

// Concatenazione con +
console.log(nome + " ha preso " + voto);
```

parseInt e parseFloat

I valori letti da un input HTML sono sempre **stringhe**, anche se sembrano numeri. **parseInt** converte in numero intero, **parseFloat** converte in numero con decimali: vanno usati prima di fare calcoli o confronti numerici su dati provenienti da un input o da un file di testo (es. CSV).

```
const testo = "7.5";
```

```
parseInt(testo); // 7 (tronca i decimali)
parseFloat(testo); // 7.5 (mantiene i decimali)

"7" + 1 // "71" ATTENZIONE: concatenazione, non somma!
parseInt("7") + 1 // 8 corretto
```

Sezione 4 — Array e Dizionari

Cos'è un array

Un array è una lista ordinata di valori, accessibili tramite un **indice numerico** che parte da 0.

```
const frutti = ["mela", "banana", "pera"];

console.log(frutti[0]); // "mela"

console.log(frutti.length); // 3
```

Metodi array

forEach — esegue un'azione per ogni elemento, non restituisce nulla di nuovo.

```
const voti = [6, 7, 8];

voti.forEach((v) => console.log(v));
```

map — trasforma ogni elemento e restituisce un nuovo array della stessa lunghezza.

```
const raddoppiati = voti.map((v) => v * 2);

// [12, 14, 16]
```

filter — restituisce un nuovo array più corto con solo gli elementi che rispettano una condizione.

```
const sufficienti = voti.filter((v) => v >= 6);

// [6, 7, 8]
```

find — come filter, ma si ferma al primo elemento trovato e restituisce quello (non un array).

```
const primoOtto = voti.find((v) => v === 8);

// 8
```

sort — riordina l'array; modifica l'array originale, quindi va usato con attenzione.

```
const numeri = [5, 1, 4, 2];

numeri.sort((a, b) => a - b); // crescente: [1, 2, 4, 5]

numeri.sort((a, b) => b - a); // decrescente: [5, 4, 2, 1]
```

reduce — riduce l'intero array a un unico valore accumulato (somma, media, massimo...).

```
const somma = voti.reduce((acc, v) => acc + v, 0);

// 21
```

```
const media = somma / voti.length;
```

Cos'è un dizionario

Un dizionario (oggetto) è una collezione di coppie **chiave: valore**. Si legge sia con la notazione a punto sia con le parentesi quadre (utile quando la chiave è dinamica, cioè contenuta in una variabile).

```
const studente = { nome: "Marco", voto: 7 };

console.log(studente.nome); // "Marco" (notazione a punto)
console.log(studente["voto"]); // 7 (parentesi quadre)

const chiave = "nome";
console.log(studente[chiave]); // "Marco" (chiave dinamica)
```

for...in, Object.keys() e Object.values()

```
const studente = { nome: "Marco", voto: 7 };

for (const chiave in studente) {
  console.log(chiave, studente[chiave]);
}

Object.keys(studente); // ["nome", "voto"]
Object.values(studente); // ["Marco", 7]
```

Array di dizionari

È la struttura standard per rappresentare dati tabellari in JavaScript: ogni elemento dell'array è un dizionario con le stesse chiavi (es. una lista di studenti).

```
const studenti = [
  { nome: "Anna", voto: 8 },
  { nome: "Marco", voto: 5 },
  { nome: "Luca", voto: 9 },
];
```

```
const promossi = studenti.filter((s) => s.voto >= 6);  
const nomiPromossi = promossi.map((s) => s.nome);  
// ["Anna", "Luca"]
```

Sezione 5 — JavaScript nel Browser (DOM)

DOM vs BOM

Il **DOM** (Document Object Model) è la rappresentazione della pagina HTML come oggetto manipolabile da JavaScript (elementi, testo, attributi). Il **BOM** (Browser Object Model) è tutto ciò che riguarda il browser stesso, non la pagina (finestra, cronologia, timer come `setTimeout`).

`document.querySelector`

Recupera un elemento della pagina tramite un selettore CSS. Si usa l'**id** preceduto da `#` perché è unico nella pagina: garantisce di selezionare esattamente quell'elemento e nessun altro.

```
<input type="text" id="nomeInput">

<script>

const campo = document.querySelector("#nomeInput");

</script>
```

`.value`: leggere il valore di un input

`.value` legge (o scrive) il contenuto di un campo input. Va sempre letto **dentro** la funzione collegata all'evento (es. dentro l'`onclick`), mai fuori: letto fuori, prenderebbe il valore presente al momento del caricamento della pagina, non quello aggiornato dall'utente.

```
const campo = document.querySelector("#nomeInput"); // fuori:
solo il gancio

bottone.onclick = function () {

const valore = campo.value; // dentro: valore aggiornato al
momento del click

console.log(valore);

};
```

`innerHTML` e `innerText`

innerHTML interpreta il testo assegnato come codice HTML (permette di inserire tag). **innerText** inserisce solo testo semplice, senza interpretare eventuali tag.

```
div.innerHTML = "<b>Ciao!</b>"; // "Ciao!" in grassetto
```

```
div.innerText = "<b>Ciao!</b>"; // mostra il testo  
"<b>Ciao!</b>" letterale
```

classList.add e classList.remove

```
elemento.classList.add("alert-success");  
elemento.classList.remove("alert-danger");
```

Eventi principali

Tag / attributo	A cosa serve
onclick	Scatta quando l'elemento (es. un pulsante) viene cliccato.
oninput	Scatta ad ogni modifica del contenuto di un campo, in tempo reale.
onkeydown	Scatta quando un tasto della tastiera viene premuto.

setTimeout e setInterval

setTimeout esegue una funzione una sola volta dopo un tempo indicato (in millisecondi). **setInterval** esegue una funzione ripetutamente ad ogni intervallo.

```
setTimeout(() => console.log("Passati 2 secondi"), 2000);  
  
setInterval(() => console.log("Ogni secondo"), 1000);
```


Sezione 6 — Evento, Azione, Reazione

Il pattern EAR

Ogni interazione con la pagina segue tre passi: **Evento** — un segnale che qualcosa è successo (un click, un submit); **Azione** — l'elaborazione dei dati in JavaScript (aggiornare un array, calcolare un valore); **Reazione** — l'aggiornamento della pagina per mostrare il nuovo stato.

Il binding

`querySelector` va chiamato **fuori** dall'`onclick` (una sola volta, per agganciare l'elemento); `.value` va letto **dentro** l'`onclick` (ad ogni click, per il valore aggiornato).

```
const campo = document.querySelector("#input"); // fuori
const bottone = document.querySelector("#bottone"); // fuori

bottone.onclick = function () {
  const valore = campo.value; // dentro
  // ...
};
```

Il pattern render()

I dati veri vivono in un array JavaScript, non nell'HTML: l'HTML è solo la "fotografia" dei dati in un certo istante. La funzione **render()** ricostruisce da zero un pezzo di pagina leggendo l'array, così l'HTML resta sempre coerente con lo stato reale, invece di aggiungere/togliere pezzi a mano.

```
// Schema base: array + render()

let studenti = [];

function render() {
  let html = "";
  studenti.forEach((s) => {
    html += `<tr><td>${s.nome}</td><td>${s.voto}</td></tr>`;
  });
  document.querySelector("#tabella").innerHTML = html;
```

```
}
```

```
render(); // disegna lo stato iniziale (vuoto)
```

Esempio completo commentato

```
const inputNome = document.querySelector("#nome");
const inputVoto = document.querySelector("#voto");
const bottone = document.querySelector("#aggiungi");

let studenti = []; // i dati veri

function render() {
  let html = "";
  studenti.forEach((s) => {
    html += `<tr><td>${s.nome}</td><td>${s.voto}</td></tr>`;
  });
  document.querySelector("#tabella").innerHTML = html; // reazione
}

bottone.onclick = function () { // evento
  const nome = inputNome.value;
  const voto = parseFloat(inputVoto.value);
  studenti.push({ nome, voto }); // azione: aggiorno i dati
  render(); // reazione: ridisegno tutto
};

render(); // stato iniziale
```

template.replace()

Alternativa al template literal: si scrive una stringa modello con dei segnaposto e si sostituiscono con `.replace()`. Utile quando il modello è lungo o riutilizzato più volte.

```
const template = "<tr><td>%NOME</td><td>%VOTO</td></tr>";  
  
const riga = template.replace("%NOME", s.nome).replace("%VOTO",  
s.voto);
```

Sezione 7 — CSV

Cos'è un CSV

CSV (Comma-Separated Values) è un formato di testo per rappresentare dati tabellari: la prima riga è l'header con i nomi delle colonne, le righe successive sono i dati, con i valori separati da virgole.

```
nome,cognome,voto
Anna,Rossi,8
Marco,Bianchi,5
```

I 4 passaggi fondamentali del parsing

1. Pulizia della stringa — `trim()` rimuove spazi/newline a inizio e fine, `replaceAll(' ', '')` rimuove eventuali spazi interni indesiderati.

```
const testo = csvGrezzo.trim().replaceAll(' ', '');
```

2. `split('\n')` — divide la stringa in un array di righe.

```
const righe = testo.split('\n');
// ["nome,cognome,voto", "Anna,Rossi,8", "Marco,Bianchi,5"]
```

3. `shift()` — estrae (e rimuove) la prima riga, l'header, dall'array.

```
const header = righe.shift().split(',');
// header: ["nome", "cognome", "voto"]
// righe ora contiene solo le righe dati
```

4. `map()` — costruisce un array di dizionari, una riga per volta, usando l'header come chiavi.

```
const studenti = righe.map((riga) => {
  const valori = riga.split(',');
  const studente = {};
  header.forEach((chiave, i) => {
    studente[chiave] = valori[i];
  });
  return studente;
});
```

Accedere ai dati dopo il parsing

```
console.log(studenti[0].nome); // "Anna"

console.log(studenti[0].voto); // "8" ATTENZIONE: è una stringa!

const voto = Number(studenti[0].voto); // 8, ora è un numero
```

Generare un CSV risultato

Percorso inverso: da un array di dizionari a una stringa CSV, con `map()` per trasformare ogni dizionario in una riga di testo e `join('\n')` per unire le righe.

```
const righeCsv = studenti.map(s =>
  Object.values(s).join(','));

const csvRisultato =
  [header.join(',')].concat(righeCsv).join('\n');
```

Il pattern `template.replace()` applicato al CSV

```
const template = "%NOME,%COGNOME,%VOTO";

const riga = template

  .replace("%NOME", s.nome)

  .replace("%COGNOME", s.cognome)

  .replace("%VOTO", s.voto);
```

Sezione 8 — AJAX e Fetch

Cos'è AJAX

AJAX permette a JavaScript di chiedere dati a un server (una API) **senza ricaricare l'intera pagina**: solo il pezzo di pagina interessato viene aggiornato, mentre il resto resta invariato.

fetch() con async/await

Servono sempre **due await**: uno per `fetch(url)` (che restituisce l'oggetto `Response`), uno per `response.json()` (che restituisce i dati veri, utilizzabili in JavaScript).

```
async function caricaDati() {  
  
  const response = await fetch(url); // 1° await: richiesta  
  
  const dati = await response.json(); // 2° await: lettura del corpo  
  
  console.log(dati);  
  
}
```

GET vs POST

	GET	POST
Quando si usa	Per leggere/richiedere dati	Per inviare/modificare dati sul server
Codice	<code>fetch(url)</code>	<code>fetch(url, { method, headers, body })</code>
Dati inviati	Nell'URL (parametri)	Nel body, come <code>JSON.stringify(...)</code>

Struttura di una HTTP request

Tag / attributo	A cosa serve
Request line	Metodo + URL + versione HTTP, es. <code>GET /users HTTP/1.1</code>
Headers	Metadati della richiesta (es. <code>Content-Type</code> , token di autenticazione).
Body	Il contenuto vero e proprio inviato (solo per POST/PUT), tipicamente JSON.

URL template con .replace()

```
const template = "https://api.esempio.it/utenti/%ID";  
  
const url = template.replace("%ID", idUtente);  
  
const response = await fetch(url);
```

Fetch GET: esempio con jsonplaceholder

```
async function caricaUtenti() {  
  const response = await  
  fetch("https://jsonplaceholder.typicode.com/users");  
  
  const utenti = await response.json();  
  
  utenti.forEach((u) => console.log(u.id, u.name, u.email));  
}
```

Fetch POST: esempio con cache remota

```
async function salvaValore(chiave, valore) {  
  const response = await  
  fetch("https://ws.cipiacinfo.it/cache/set", {  
  
    method: "POST",  
  
    headers: {  
  
      "Content-Type": "application/json",  
  
      "key": TOKEN, // il token va nell'header, non nel body  
  
    },  
  
    body: JSON.stringify({ key: chiave, value:  
      JSON.stringify(valore) }),  
  
  });  
  
  const risultato = await response.json();  
  
  console.log(risultato);  
}
```

Sezione 9 — Cache Remota

Cos'è la cache remota

La cache remota è un piccolo "armadietto condiviso" su un server: permette di salvare e rileggere coppie chiave-valore da qualsiasi pagina, utile per far sopravvivere dati oltre il singolo caricamento della pagina (es. una lista di preferiti).

Il token

Il token identifica chi sta usando la cache (per evitare che utenti diversi si sovrascrivano i dati a vicenda). Va messo nell'**header** personalizzato key della richiesta, non nel body.

SET: salvare una coppia chiave-valore

```
async function set(chiave, valore) {  
  
  const response = await  
  fetch("https://ws.cipiaceinfo.it/cache/set", {  
  
    method: "POST",  
  
    headers: {  
  
      "Content-Type": "application/json",  
  
      "key": TOKEN,  
  
    },  
  
    body: JSON.stringify({ key: chiave, value:  
      JSON.stringify(valore) }),  
  
  });  
  
  return await response.json();  
  
}
```


GET: leggere un valore dalla chiave

```
async function get(chiave) {  
  const response = await  
    fetch("https://ws.cipiaceinfo.it/cache/get", {  
      method: "POST",  
      headers: {  
        "Content-Type": "application/json",  
        "key": TOKEN,  
      },  
      body: JSON.stringify({ key: chiave }),  
    });  
  const dati = await response.json();  
  
  if (typeof dati.result === "string") {  
    return JSON.parse(dati.result); // chiave trovata  
  }  
  return null; // chiave non trovata (dati.result è un oggetto di  
    errore)  
}
```

Perché anche GET usa POST

Anche la lettura usa il metodo POST perché bisogna **inviare la chiave da cercare** nel body della richiesta: con una semplice GET non ci sarebbe modo standard di allegare quel dato senza metterlo nell'URL.

JSON.stringify prima di salvare, JSON.parse dopo aver letto

La cache remota salva solo **stringhe**. Per salvare un dato complesso (array, dizionario) va prima convertito in stringa con `JSON.stringify`; dopo averlo riletto, va riconvertito in dato utilizzabile con `JSON.parse`.

Esempio completo: salvare e rileggere un array

```
const preferiti = [{ id: 1, nome: "Film A" }, { id: 2, nome:
"Film B" }];

// Salvataggio

await set("preferiti-utente", preferiti);

// Rilettura

const listaLetta = await get("preferiti-utente");

console.log(listaLetta);

// [{ id: 1, nome: "Film A" }, { id: 2, nome: "Film B" }]
```